SITS 2026

奇点智能技术大会

Singularity Intelligence Technology Summit

主办方 / Organizers：奇点智能研究院 Singularity Intelligence Research Institute　CSDN

合作伙伴 / Partners：CECW 中能拾贝　OMNI-AI　乐奇 Rokid　异步社区 www.epubit.com　清華大學出版社 TSINGHUA UNIVERSITY PRESS

2026技术大会活动预告

奇点智能研究院 Singularity Intelligence Research Institute CSDN

SITS 2026

奇点智能技术大会

Singularity Intelligence Technology Summit

4月17-18日·上海 | 9月11-12日·北京

PM – Summit 2026

全球产品经理大会

Product Management Summit

7月17-18日·北京

CPP – Summit 2026

全球系统软件技术大会

C++ and System Software Summit

11月20-21日·上海

GOSIM®

GOSIM 2026

5月4-6日·巴黎 | 10月16-18日·深圳

2026年度 1024程序员日

智汇潮湃"人工智能+"创新大赛

AI构建世界 智能引领未来

10月24-25日·多城联动

媒体服务——多形态媒体矩阵

奇点智能研究院 Singularity Intelligence Research Institute　CSDN

为客户提供定制化内容服务，包括但不限于视频、文章、评测、专题报告、白皮书等多品类内容

访谈 → 新闻稿件 → 产品解读 → 产品评测 → 客户案例 → 活动报道 → 专题

直播栏目

《AI进化论》

《涛滔不绝》

《万有引力》

文章栏目

《AGI技术50人》

《近匠》

《开源英雄》

权威出版

《新程序员》

聚集全球技术大师、行业技术先锋，从前沿技术到行业先锋，为您带来全方位的深刻洞见和专业解读。

《专题报告》

《AI原生软件研发成熟度模型AISMM》
《2025全球开源发展报告》
云原生、AI赋能软件研发、数据库、操作系统等领域报告

技术咨询——企业信赖的策略伙伴

奇点智能研究院 Singularity Intelligence Research Institute CSDN

依托雄厚的专家团队和丰富的行业经验，通过12大技术咨询板块，提供涵盖人工智能、架构设计、产品管理等全栈技术咨询解决方案，帮助企业在数字化、智能化、万物互联的变革中打造关键创新能力，加速技术和产品迭代升级。

6大核心研究领域

01 智能化软件工程与研发效能

02 AI系统软件技术栈

03 企业开源软件设计与治理

04 软件架构设计与重构

05 系统软件性能工程与优化

06 智能化战略转型与落地

SITS 2026

奇点智能技术大会

Singularity Intelligence Technology Summit

演讲主题 Topic

Toward Self-Improving Agents：构建可自我增强的Agent工程体系

王乐业

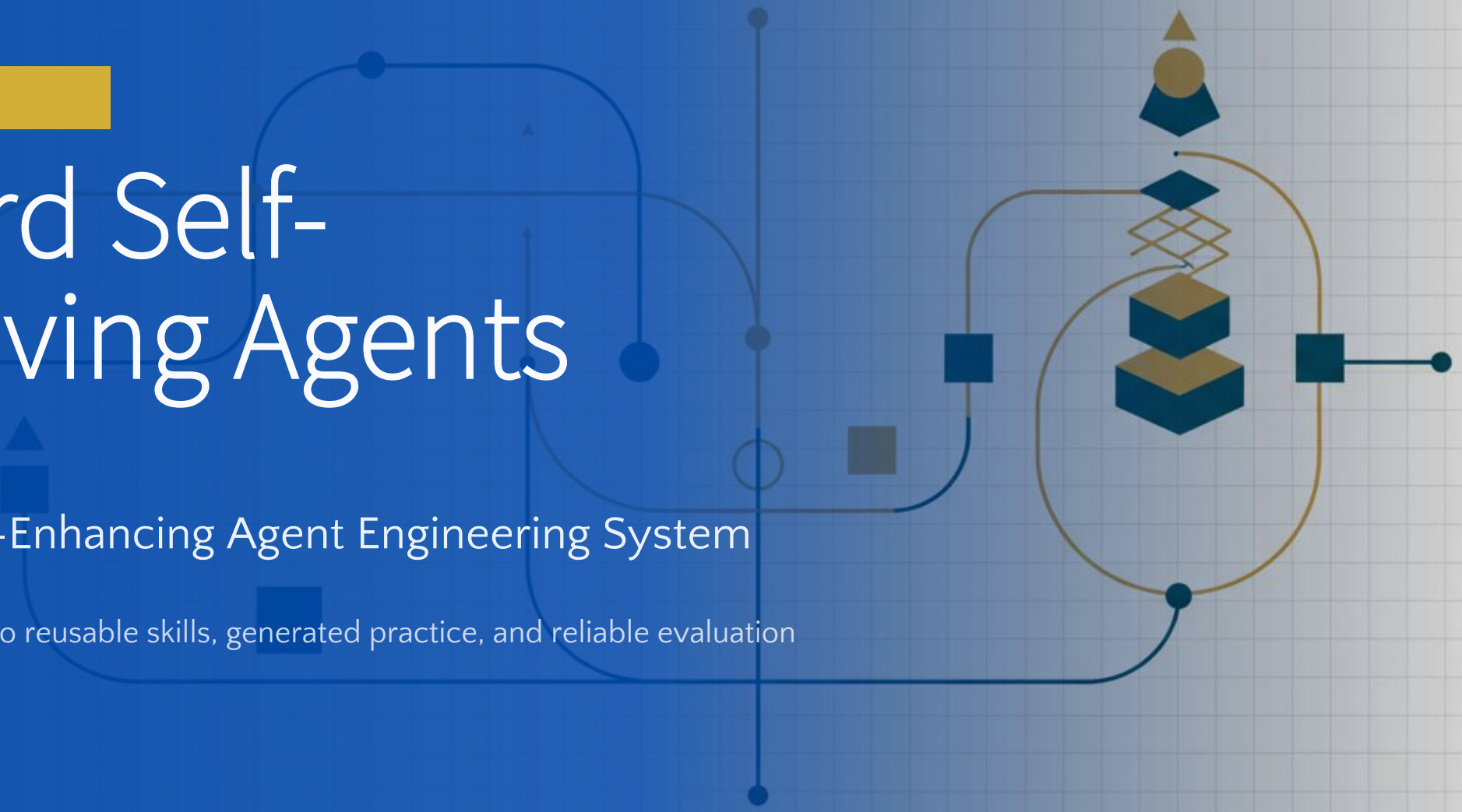
北京大学计算机学院长聘副教授

AGENT ENGINEERING

Toward Self-Improving Agents

Building a Self-Enhancing Agent Engineering System

From task correction to reusable skills, generated practice, and reliable evaluation



PARADIGM SHIFT

Why Self-Improving Agents, and Why Now?

The Discussion Is Shifting

From **"Can agents complete tasks?"** to **"Can agents improve over time?"**

The fundamental question in agent research has evolved from static capability to dynamic learning and adaptation.

Recent Systems Highlight

- **Learning loops** — continuous feedback cycles
- **Skill accumulation** — building reusable capabilities
- **Persistent adaptation** — memory across sessions

Self-improvement is becoming a core aspiration of agent systems

The Trend Is Clear

Static Execution →

Continuous Improvement ↑

Agent focus is shifting from one-time task completion to sustained capability enhancement

CONCRETE SIGNAL

A Concrete Signal: Hermes Agent

> Hermes Agent is explicitly positioned as a self-improving AI agent
>
> Its official description emphasizes built-in capabilities for continuous learning and adaptation from experience.

Core Self-Improvement Features

Built-in learning loop
Continuous feedback integration

Creating skills from experience
Automated skill extraction

Improving skills during use
Runtime refinement

Persistent knowledge
Cross-session memory

Self-Evolution Project

A separate Hermes project explores automated evolution of **skills**, **prompts**, **tools**, and **code** using DSPy + GEPA.

Official Positioning

"The agent that grows with you"
Self-description from official repository

Self-Evolution Repository
nousresearch/hermes-agent

What This Signals

"Self-improving" is transitioning from research concept to **system narrative** and **product positioning**

The Opportunity Is Real — But the Problem Is Still Underspecified

"Self-Improving" Is Promising, But...

The concept remains conceptually mixed. Different aspects are often discussed together without clear boundaries or decomposition.

Different Things Often Mixed Together

Memory
Storing context

Retry
Re-execution

Skill Accumulation
Capability building

Skill Evolution
Refinement over time

Task Generation
Practice creation

Evaluation
Quality assessment

What Is Missing: A Cleaner Problem Decomposition

We need to separate concerns and define clear boundaries between different aspects of self-improvement.

The Challenge

Conceptual Clarity
Distinguishing different mechanisms

Engineering Focus
Building operational systems

Clear Boundaries
Defining component responsibilities

CORE FRAMEWORK

Three Core Questions

These questions drive the entire framework and guide our engineering approach

01 Who teaches the agent to improve?

Identifying the source of improvement guidance — is it self-directed, mentor-guided, or evidence-driven?

02 What artifact is actually improved?

Defining the output of improvement — prompts, skills, code, or system configuration?

03 When does a local fix become real capability gain?

Establishing criteria for recognizing transferable, reusable improvement vs. one-time patches.

All subsequent content addresses these three fundamental questions

CORE POSITION

Our Position: Verified and Reusable Improvement

A Single Successful Retry Is NOT Enough

Many systems conflate **temporary fixes** with **genuine improvement**. We need stricter criteria.

Real Self-Improvement Must Satisfy Two Properties

Verified

Objectively measured, not assumed. Clear evaluation criteria and measurable outcomes.

Reusable

Transfer to new contexts, not just fix one task. Generalizable patterns and explicit artifacts.

The Goal: Transferable Capability Accumulation

Not merely fixing one task, but building reusable, composable capabilities that compound over time.

The Distinction

Local Fix

- Task-specific patch
- Implicit knowledge
- Hard to reproduce
- No accumulation

Real Improvement

- Reusable pattern
- Explicit skill
- Transferable
- Compounds over time

TEACHER IDENTIFICATION

Who Is the Teacher?

Failure Traces Alone Do Not Teach

They provide **evidence** of what went wrong, but not reliable **abstraction** of how to improve. Raw error logs don't automatically translate to actionable guidance.

In SkillMentor: Three Components

Stronger Mentor Model

Provides **improvement guidance** through better diagnosis and abstraction capabilities

Verification

Provides **trustworthy judgment** about whether improvement actually occurred

Failure Traces

Provide **diagnostic evidence** about what happened and where things went wrong

Key Insight

"Failure trace is evidence, not the teacher"

The teacher is the stronger mentor model that provides improvement guidance

The Relationship

1. **Evidence** (failure traces)
2. **Teacher** (mentor model)
3. **Validation** (verification)

MINIMAL FRAMEWORK

A Minimal Decomposition for Self-Improving Agent Engineering

Evidence

What happened?

Capturing execution traces, outputs, failures, and behavioral data

Teacher

What should change?

Providing guidance, diagnosis, and actionable improvement strategies

Validation

Did it really improve?

Verifying that changes actually lead to measurable capability gains

SkillMentor's Focus

SkillMentor focuses on the **first operational loop** that combines all three components into a working system. This is the foundational building block for self-improving agent engineering.

Evidence Collection
Automated trace capture

Teacher-Guided Review
Mentor model analysis

Validation Loop
Verification & testing

STARTING POINT

Why Start from SkillMentor?

Failure Correction: The Most Tractable Loop

Among all improvement mechanisms, **failure correction** offers the clearest entry point for building operational self-improving systems.

Easy to Observe
Clear failure signals

Easy to Verify
Binary pass/fail criteria

Easy to Operationalize
Concrete engineering path

A Concrete Path from Failure to Skill

Failed Execution → **Analysis** → **Guided Retry** → **Reusable Skill**

Most Tractable, Most Verifiable, Most Like Engineering

Why Not Full Evolution?

Online Self-Evolution

- Complex to observe
- Hard to verify
- Difficult to debug
- Engineering challenge

Failure Correction

- Clear signals
- Binary verification
- Observable process
- Engineering-ready

ONE-SENTENCE DEFINITION

SkillMentor in One Sentence

A review-driven runtime for turning failure correction into reusable skills

1 Weak Agent Attempts First

The baseline agent executes the task without prior intervention, generating authentic failure traces for analysis.

2 Mentor Guides After Verification

A stronger mentor model provides improvement guidance only after verification identifies actual failures.

3 Patterns Become Skill Candidates

Reusable correction patterns are extracted and formalized as explicit, manageable skill artifacts.

Weak Agent
Baseline execution

Verification
Failure detection

Mentor Review
Guided improvement

Skill Formation
Reusable artifact

PROBLEM ANALYSIS

The Limitation of Ordinary Retry

Ordinary Retry Optimizes the Current Task

Standard retry mechanisms focus on **fixing this specific instance**, not on building lasting capability. They produce local optimizations without systemic improvement.

Without Structure, Improvement Remains:

Implicit
Buried in conversation history

Fragile
Breaks with slight variations

Hard to Reuse
Not transferable to new tasks

Hard to Evaluate
No clear success criteria

> "Not all retries are learning"

This is why we need mentor guidance, verification, and explicit skill extraction.

The Difference

Ordinary Retry

- Fix this instance
- No structure
- No accumulation
- Hard to verify

SkillMentor Approach

- Build capability
- Structured process
- Skill formation
- Verified improvement

KEY IDEA

The Key Idea of SkillMentor

From Trace-Only Reflection to Teacher-Guided Improvement

Instead of relying solely on the agent's own reflection on traces, we introduce a **stronger mentor model** that provides explicit improvement guidance based on better diagnosis capabilities.

From One-Off Patching to Reusable Skill Formation

Rather than producing task-specific patches, we extract **generalizable patterns** and formalize them as explicit, reusable skills that can be applied to future tasks.

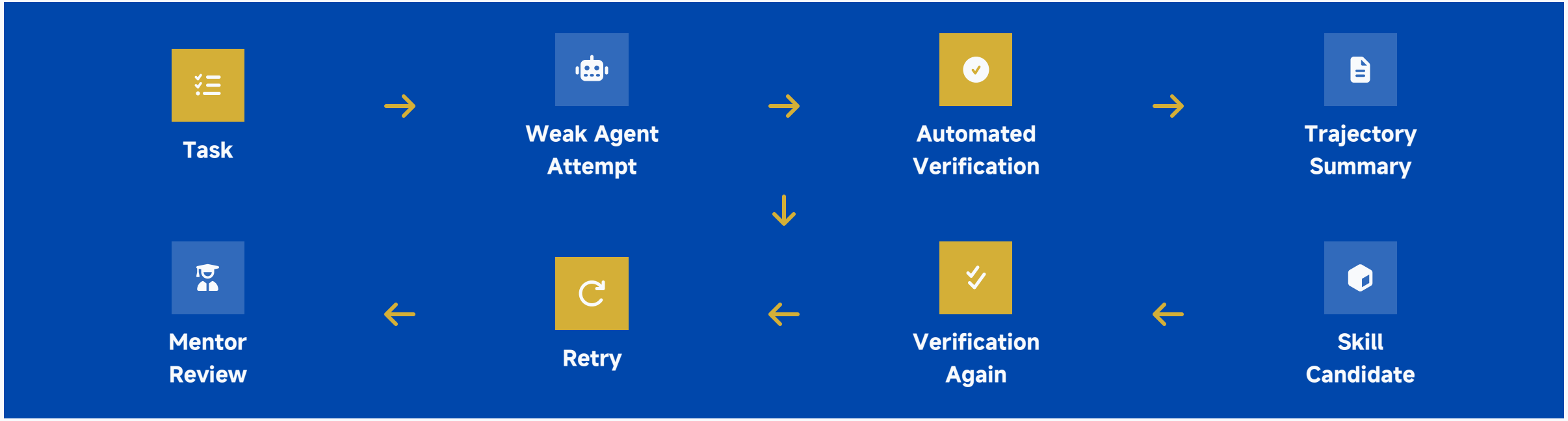
From Subjective Feeling to Verification-Backed Improvement

Instead of assuming improvement, we **measure it** through automated verification, task checks, and quality judgments to ensure real capability gains.

Patch ≠ Skill

OPERATIONAL WORKFLOW

The Runtime Loop



Key Point 1: Mentor Timing

The **mentor does NOT intervene** before the first attempt. The weak agent tries first to generate authentic failure traces for analysis.

Key Point 2: Conditional Skill Creation

Skill creation is **optional and conditional** — only when the correction pattern is judged reusable and the improvement is verified.

ROLE CLARIFICATION

Three Roles: Weak Agent, Judge, Mentor

Clear separation of responsibilities is essential for systematic improvement

Weak Agent

Executes the Task

Attempts to complete assigned tasks with baseline capabilities

Object Being Improved

The target of improvement efforts and skill application

Judge / Verifier

Checks Constraints

Validates hard constraints and quality criteria objectively

No Correction Strategy

Does NOT decide how to fix — only identifies what failed

Mentor

Diagnoses Failures

Analyzes traces to identify root causes and patterns

Proposes Guidance

Creates actionable retry guidance and identifies reusable patterns

⚠ **Critical Distinction:** Judge and mentor are NOT the same. The judge identifies **whether** something failed; the mentor determines **how** to improve.

GROUNDING MECHANISM

How Improvement Is Grounded

Mapping the three-component framework onto SkillMentor's implementation

Evidence

What happened?

- **Outputs** — generated content
- **Task-check failures** — constraint violations
- **Logs and traces** — execution history

Teacher

What should change?

- **Stronger mentor model** — better capabilities
- **Better diagnosis** — root cause analysis
- **Abstraction** — pattern recognition

Validation

Did it improve?

- **task_check** — hard constraints
- **model_judge** — quality scoring
- **Skill testing** — reuse validation

Complete Loop: Evidence → Teacher → Validation → Skill Formation

FEEDBACK STRUCTURE

Why Structured Mentor Feedback Matters

Unstructured Critique Is Hard to Reuse

Free-form feedback like "this could be better" or "fix the structure" doesn't provide actionable guidance. It's difficult to parse, validate, or transform into reusable artifacts.

To Support Accumulation, Feedback Must Be:

Explicit

Clearly stated problems and actions

Structured

Organized into defined categories

SkillMentor Organizes Output Into:

- **Diagnosis**
- **Evidence**
- **Action**
- **Reuse Signal**

Why Structure Matters

For Engineering

Enables automated processing and skill extraction

For Verification

Allows validation of whether guidance was followed

For Accumulation

Creates reusable, composable skill artifacts

MENTOR OUTPUT

What the Mentor Actually Produces

Diagnosis

Summary
High-level description of what went wrong

Failure Type
Categorization (e.g., missing info, structure)

Root Cause
Underlying reason for the failure

Action

Recommended Action
What should be done differently

Retry Guidance
Specific, executable instructions

Implementation
How to apply the guidance

Reuse Signal

Trigger Signals
When to apply this pattern

Skill Candidate
Suggestion for skill extraction

Generalization
How pattern transfers to other tasks

Key Principle: The mentor doesn't just say what's wrong — it provides **structured, actionable guidance** that can be verified, executed, and potentially transformed into reusable skills.

GUIDANCE PRINCIPLE

From Diagnosis to Retry Guidance

The Mentor Does NOT Simply Rewrite

Instead of directly producing the final corrected output, the mentor generates **executable guidance** that the weak agent can follow. This preserves the learning and improvement process.

This Approach Preserves:

Modularity

Clear separation between diagnosis, guidance, and execution

Reusability

Guidance can be applied to similar situations

Generalization

Patterns can transfer to new contexts

Retry Guidance: Bridge from Analysis to Execution

Retry guidance connects the mentor's diagnosis with the weak agent's improved execution, enabling **teacher-guided learning** rather than direct replacement.

Why Not Direct Rewrite?

Preserves Learning

Weak agent still executes and learns

Enables Verification

Can check if guidance was followed

Supports Reuse

Guidance applies to similar tasks

Builds Capability

Agent improves, not just output

VERIFICATION DESIGN

Why Verification Is Split into task_check and model_judge

task_check

Validates Hard Constraints

Checks objective, binary criteria (e.g., required sections present)

Primary Gating Signal

Determines if output meets minimum requirements

model_judge

Scores Quality Dimensions

Evaluates grounding, coverage, structure, and overall quality

Provides Measurement

Quantifies improvement and identifies specific issues

The Combination Avoids Both Extremes

Purely Formal Passing

task_check alone might miss quality issues — model_judge catches these

Purely Subjective Evaluation

model_judge alone might miss constraint violations — task_check ensures compliance

Design Logic: Hard constraints and quality judgment are **separated** to create a more robust, reliable verification system that catches different types of failures.

IMPROVEMENT CRITERIA

When Does Improvement Count?

Improvement Must Be Verified

Subjective feelings of improvement are insufficient. We need **objective, measurable criteria** to recognize genuine capability gains.

MVP Recognition Criteria:

Binary Pass

task_check changes from fail to pass — objective constraint satisfaction

Score Improvement

Judge scores improve meaningfully — quality dimensions show gains

But Verified Improvement Alone Is Still NOT Enough

We must also ask: **Is it reusable?** Verified improvement is **necessary but not sufficient** for real self-improvement.

The Two Requirements

1. Verified

Measurable, objective improvement

2. Reusable

Transferable to new contexts

Both required for real self-improvement

SKILL PROMOTION

When Does a Retry Become a Skill Candidate?

Retry Gain
Measurable improvement

→

Reusable Pattern
Mentor judges generality

→

Skill Candidate
Promoted to explicit skill

1 Measurable Improvement

The retry must show **concrete, measurable improvement** through verification — either task_check passes or judge scores improve significantly.

2 Reusable Pattern

The mentor must **judge the correction pattern as reusable** — not just a task-specific fix, but a generalizable approach applicable to similar situations.

3 Not Task-Specific

The correction must **not be merely task-specific** — it should address a category of problems rather than a single instance.

Clear Logic Chain: Retry Gain → Reusable Pattern → Skill Candidate — only then is improvement promoted into the skill library.

SKILL
ARTIFACT

What Gets Written as a Skill?

SKILL.md

Executable skill specification with structured guidance

When to Use
Trigger conditions and applicability criteria

Do Not Use
Constraints and anti-patterns to avoid

Steps
Actionable, ordered guidance for execution

Checks
Verification criteria for each step

Success Criteria
Definition of successful application

meta.yaml

Metadata for provenance, tracking, and management

Provenance
Origin and creation context

Source Trajectory
Execution history that generated this skill

Mentor Feedback
Structured diagnosis and guidance

Models
Versions and configurations used

Verification Metadata
Test results and quality scores

Key Principle: A skill is not just "lesson learned" — it's an **explicit, executable, verifiable, and manageable** artifact that can be tested, versioned, and evolved.

RATIONALE

Why Explicit Skills Matter

Alternatives Are Insufficient:

Conversation History

Too **implicit** — buried in context, hard to extract, difficult to reuse

Prompt Patches

Too **brittle** — task-specific, break easily, don't generalize

One-Off Reflections

Too **hard to transfer** — instance-specific, not operationalizable

Explicit Skills Enable:

- **Reuse**
- **Management**
- **Testing**
- **Evolution**

Paving the Way

For SkillFab

Skill development and lifecycle management

For SkillTester

Continuous testing and selection

For Full Stack

Complete self-improving system

CASE STUDY SETUP

A Concrete Example

Task: Generate Annual-Report Analysis Webpage

A complex content generation task requiring structured analysis, evidence grounding, and clear presentation — ideal for demonstrating SkillMentor's correction and skill extraction capabilities.

Common First-Attempt Problems:

Missing Sections
Required content areas omitted

Weak Grounding
Insufficient evidence support

Generic Conclusions
Lack of specific insights

Poor Separation
Growth and risk not distinguished

Why This Example: These problems make it ideal for demonstrating **structured correction** and **skill extraction** — clear failures, verifiable improvements, and reusable patterns.

Task Characteristics

Complex
Multiple requirements and constraints

Observable
Clear success/failure criteria

Verifiable
Binary and quality checks apply

Reusable Patterns
Corrections apply to similar tasks

VISUAL COMPARISON

Before → Mentor → After

Before

- Incomplete structure
- Missing sections
- Weak evidence grounding
- Generic conclusions

Task Check: FAIL

Mentor

Identifies Root Cause
Structure-first approach missing

Proposes Guidance
Section-first retry strategy

Reusable Pattern
Generalizable to similar tasks

After

- Better completeness
- All sections present
- Stronger grounding
- Specific insights

Task Check: PASS

Outcome

A reusable skill candidate is extracted

Section-first structure guidance
Applicable to analysis tasks

VALUE SUMMARY

What SkillMentor Really Contributes

Not Just Retry

Goes beyond simple re-execution to structured, guided improvement with verification

Not Just Critique

Provides actionable, executable guidance rather than unstructured feedback

Not Just Prompt Refinement

Builds reusable skills, not brittle patches or temporary fixes

A Minimal but Complete Loop For:

Teacher-Guided Correction

Stronger mentor model provides improvement guidance based on diagnosis

Verification-Backed Improvement

Objective measurement ensures real capability gains, not assumed progress

Skill-Oriented Accumulation

Reusable patterns become explicit, manageable, testable skill artifacts

SkillMentor turns local correction into a path toward reusable capability

SCOPE ACKNOWLEDGMENT

SkillMentor Is Only the First Loop

What SkillMentor Answers Well

How to learn from failed tasks with stronger guidance — a concrete, operational, and verifiable improvement loop that turns failure correction into reusable skills.

But a Full Self-Improving Stack Still Needs:

Better Task Generation

What should the agent practice on?

Skill Lifecycle Management

How should skills be developed?

Continuous Testing

How do we know it's improving?

Our Approach: First, make the "learning from failure" loop **operational and robust**. Then expand to the full self-improving stack.

Building Blocks

✓ **SkillMentor**

Learning from failure (DONE)

○ **Task Generation**

What to practice (NEXT)

○ **SkillFab**

Skill development (FUTURE)

○ **SkillTester**

Continuous testing (FUTURE)

TRANSITION

Transition to the Full Stack

If SkillMentor tells us how to learn from failure, the next questions are:

1. What should the agent practice on?
2. How should skills be developed and managed?
3. How do we know the agent is really improving over time?

The journey toward self-improving agents continues...

AI生成